



O.1 STUDENT DETAILS

Student Name: OTIENO LAZARO

Admission Number: BSCCS/2024/74720

Course: COMPUTER SCIENCE

Unit Code: BIT3208

Unit Name: ADVANCED WEB DESIGN AND DEVELOPMENT

Lecturer Name: MICHAEL NYORO

Semester: 3.2

0.2 PROJECT DETAILS

Project Title: E-Commerce System

Tech Stack: PHP, SQL, CSS, JS

0.3 GITHUB LINK

LINK: <https://github.com/Laz-28/techstore-ecommerce>

Table of Contents:

Week 1: Local Environment Setup	2
Title: Installation and Testing of Local Development Environment.....	3
Required Evidence:	3
Fig 1: Installation of XAMPP.	3
Fig 2: Apache and MySQL Running.	3
Fig 3: Localhost Test Page.	3
Fig 4: Hello World Test.....	4
Fig 5: Database Connection Test.....	4
Student Reflection:	4
Week 2: Wireframes and GUI Design	5
Title: User Interface Planning and System Design	5
Required Evidence:	5
Fig 1: Hand-drawn Wireframe.	5
Fig 2: Dashboard Layout Design.....	5
Fig 3: Mobile View Mockup.	5
Fig 4: Color Theme and Navigation Structure.....	6
Fig 5: GUI Prototype.	6
Student Reflection:	7
Week 3: JavaScript and PHP Basics	7
Title: Frontend Interaction and Backend Foundations	8
Required Evidence:	8
Fig 1: JavaScript Form Validation/Interaction.	8
Fig 2: Secure Password Handling.	8
Fig 3: PHP Syntax Practice.	9
Fig 4: Database Connection Script.	9
Fig 5: Dynamic User Input Handling.....	9
Student Reflection:	10
Week 4: Server-Side Components and Backend Development Foundations	10

Required Evidence:	11
Fig 4.1: Modular Folder Structure.	11
Fig 4.2: PHP Form Processing.	11
Student Reflection:	12
Week 5: Database Components and CRUD Operations	12
Implementing Secure Database Management and Access Control	12
Required Evidence	12
Fig 5.1: Database Schema Expansion.	12
Fig 5.2: The CRUD Administrative Panel.....	12
Fig 5.3: Role-Based UI Rendering.	13
Fig 5.4: Secure Server Logic.	13
Student Reflection:	13

Week 1: Local Environment Setup

Title: Installation and Testing of Local Development Environment

Required Evidence:

Fig 1: Installation of XAMPP.



Fig 2: Apache and MySQL Running.

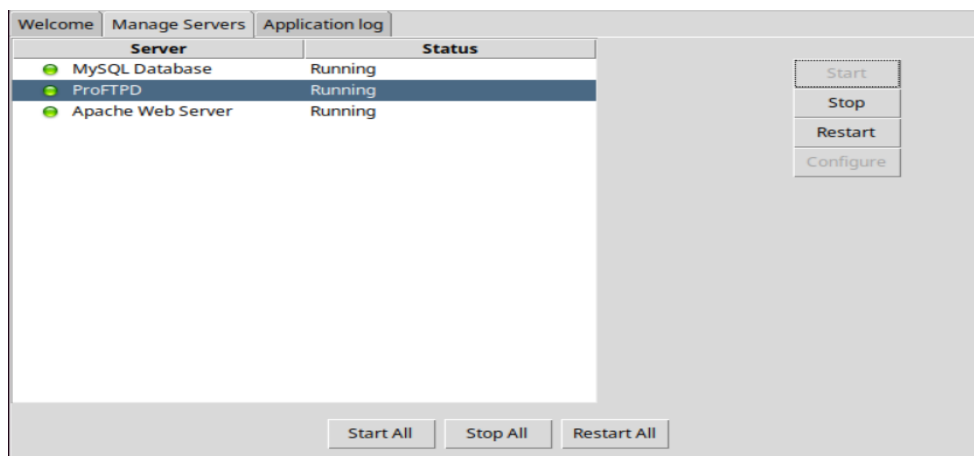


Fig 3: Localhost Test Page.

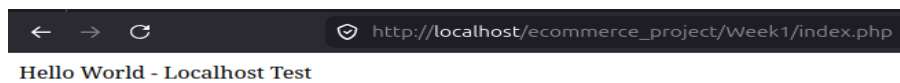


Fig 4: Hello World Test.

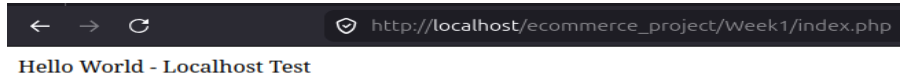


Fig 5: Database Connection Test.

		id	username	email	password	created_at
☐	Edit	1	laz	otienolazaro111@gmail.com	\$2y\$10\$vDjzDmZOQfLFCMPBr0L7OogrLsnCxjRvw/wCQEWXw...	2026-05-21 06:53:12
☐	Edit	2	Judi	judi@gmail.com	\$2y\$10\$UW0wN1aGQVD1KukXGpkj7.R8C8cwtFqZVynR8p7Qx4...	2026-05-21 07:43:00
☐	Edit	3	Mike	mike@gmail.com	\$2y\$10\$Oqi3dO5WuPAjDI.1phgZve7N37.0mE9dHdF1fhFtCOM...	2026-05-21 10:37:44

Student Reflection:

Fig 5 shows the successful configuration of the MySQL database (ecommerce_project) using XAMPP, establishing the foundation for the TechStore backend. I learned how to set up local Apache and MySQL services to host server-side scripts. The primary challenge was ensuring the db.php PDO connection string correctly targeted the local database without throwing exceptions. These screenshots demonstrate a fully functional local environment ready for dynamic PHP rendering and secure database queries.

GitHub: <https://github.com/Laz-28/techstore-ecommerce>

Week 2: Wireframes and GUI Design

Title: User Interface Planning and System Design

Required Evidence:

Fig 1: Hand-drawn Wireframe.

The wireframe illustrates a user authentication interface with two main sections: account creation and login. The 'Create an account' section includes fields for Username, Email (pre-filled with 'balaria@gmail.com'), and Password, followed by a 'Create account' button and a 'Continue with Google' button. A link to 'Log In' is provided for existing users. The 'Login to your account' section includes fields for Email (pre-filled with 'balaria@gmail.com') and Password, with a 'Forgot ?' link next to the password field. A 'Login now' button is at the bottom, with a 'Sign Up' link for users without an account. The entire form is set against a light gray background with rounded corners.

Create an account

UserName

Email

Password

[Create account](#)

[Continue with Google](#)

Already Have An Account ? [Log In](#)

Login to your account

Email

Password [Forgot ?](#)

[Login now](#)

Don't Have An Account? [Sign Up](#)

Fig 2: Dashboard Layout Design.

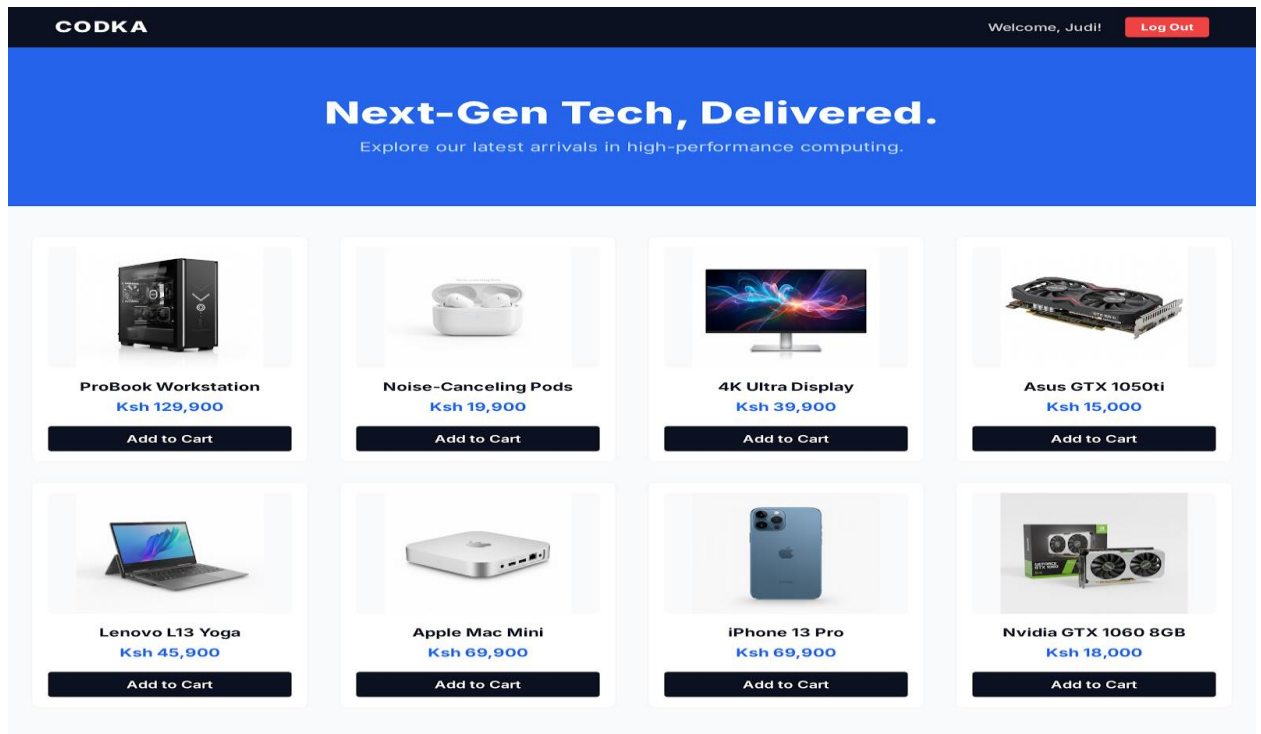


Fig 3: Mobile View Mockup.

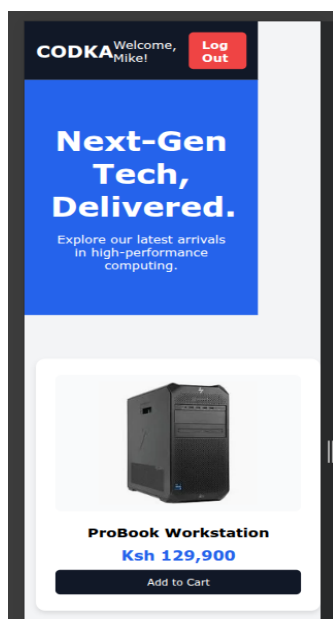


Fig 4: Color Theme and Navigation Structure.

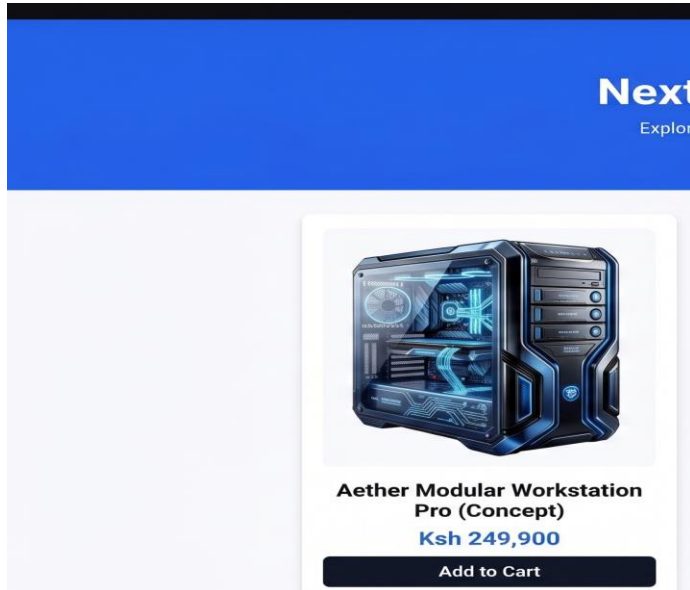
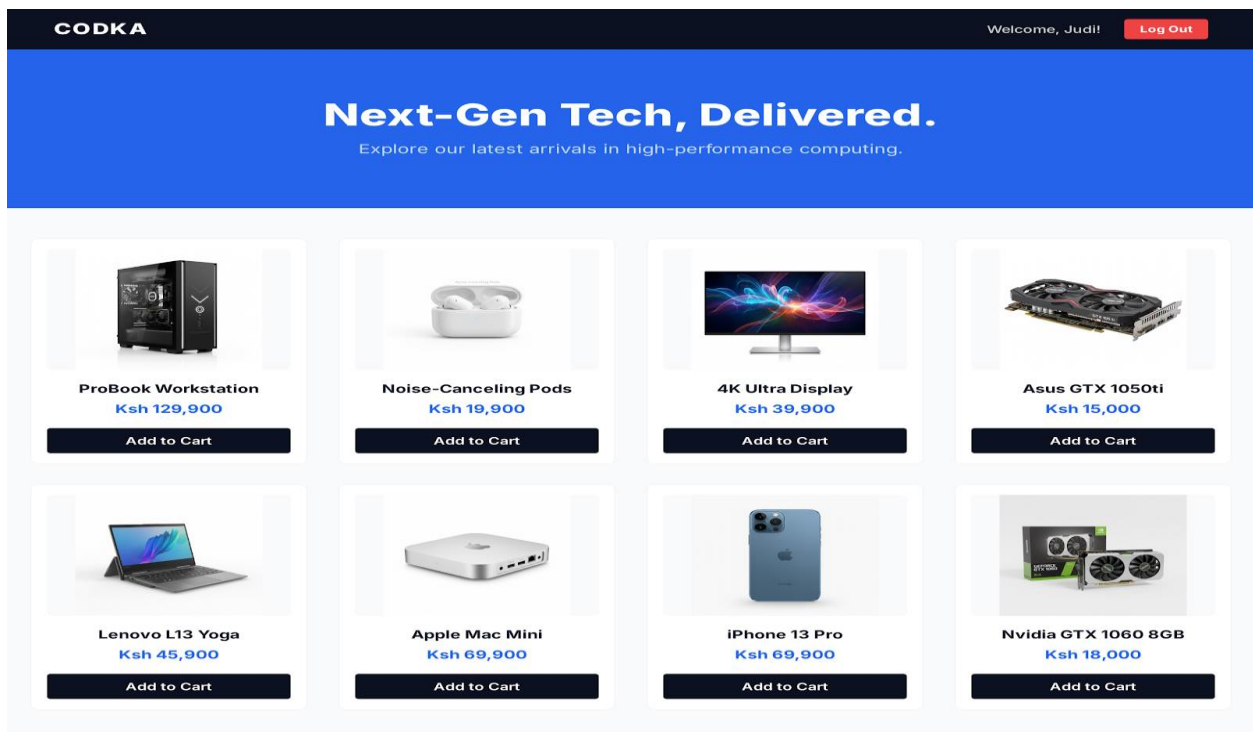


Fig 5: GUI Prototype.



Student Reflection: The TechStore UI was designed utilizing Flexbox to ensure a responsive, mobile-first experience. The centered card layout reduces cognitive load during authentication. A clean, dual-state toggle was planned for the Login/Signup flow to keep the interface contained within a single page view, improving the overall user experience. The color palette relies on

high-contrast blues and neutral grays to establish a professional, modern e-commerce aesthetic suitable for the target demographic.

Github: <https://github.com/Laz-28/techstore-ecommerce/commit/8ee458ec0b5116e91228ce2958b7e156bf722d30>

Week 3: JavaScript and PHP Basics

Title: Frontend Interaction and Backend Foundations

Required Evidence:

Fig 1: JavaScript Form Validation/Interaction.

```
const loginView = document.getElementById('login-view');
const signupView = document.getElementById('signup-view');
const goToSignup = document.getElementById('go-to-signup');
const goToLogin = document.getElementById('go-to-login');

goToSignup.addEventListener('click', function(e) {
    e.preventDefault();
    loginView.classList.add('hidden');
    signupView.classList.remove('hidden');
});

goToLogin.addEventListener('click', function(e) {
    e.preventDefault();
    signupView.classList.add('hidden');
    loginView.classList.remove('hidden');
});
```

Fig 2: Secure Password Handling.

```
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['action']) && $_POST['action'] === 'register') {
    $username = $_POST['username'];
    $email = $_POST['email'];
    $password = password_hash($_POST['password'], PASSWORD_DEFAULT);

    $stmt = $pdo->prepare("INSERT INTO users (username, email, password) VALUES (?, ?, ?)");
    try {
        $stmt->execute([$username, $email, $password]);
        $_SESSION['user_id'] = $pdo->lastInsertId();
        $_SESSION['username'] = $username;
        header("Location: landing.php");
        exit();
    } catch (PDOException $e) {
        die("Registration failed. Email might already exist.");
    }
}

if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['action']) && $_POST['action'] === 'login') {
    $email = $_POST['email'];
    $password = $_POST['password'];

    $stmt = $pdo->prepare("SELECT * FROM users WHERE email = ?");
    $stmt->execute([$email]);
    $user = $stmt->fetch();

    if ($user && password_verify($password, $user['password'])) {
        $_SESSION['user_id'] = $user['id'];
        $_SESSION['username'] = $user['username'];
        header("Location: landing.php");
        exit();
    } else {
        die("Invalid email or password.");
    }
}
```

Fig 3: PHP Syntax Practice.

```
<?php

session_start();
require 'db.php';

if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['action']) && $_POST['action'] === 'register') {
    $username = $_POST['username'];
    $email = $_POST['email'];
    $password = password_hash($_POST['password'], PASSWORD_DEFAULT);

    $stmt = $pdo->prepare("INSERT INTO users (username, email, password) VALUES (?, ?, ?)");
    try {
        $stmt->execute([$username, $email, $password]);
        $_SESSION['user_id'] = $pdo->lastInsertId();
        $_SESSION['username'] = $username;
        header("Location: landing.php");
        exit();
    } catch (PDOException $e) {
        die("Registration failed. Email might already exist.");
    }
}
```

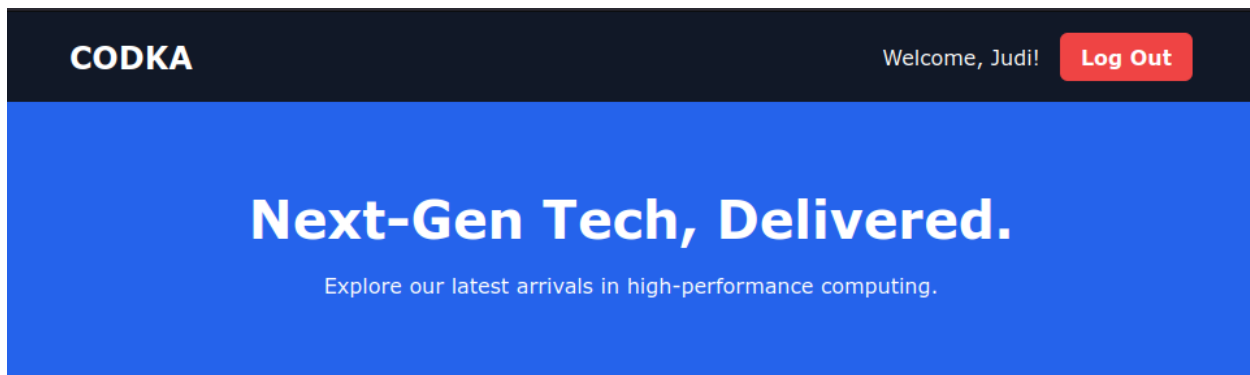
Fig 4: Database Connection Script.

```
<?php
$host = 'localhost';
$dbname = 'ecommerce_project';
$db_user = 'root';
$db_pass = '';

$dsn = "mysql:host=$host;dbname=$dbname;charset=utf8mb4";

try {
    $pdo = new PDO($dsn, $db_user, $db_pass);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Database connection failed: " . $e->getMessage());
}
?>
```

Fig 5: Dynamic User Input Handling.



Student Reflection: This week integrated frontend manipulation with backend processing. Fig 1 demonstrates vanilla JavaScript manipulating the DOM to seamlessly toggle between authentication states without a page reload. Fig 3 and 4 show how auth.php captures the POST request, securely hashes the password, and interacts with MySQL via PDO. Instead of relying on client-side cookies, server-side sessions (session_start()) were successfully implemented to authenticate the user and dynamically render their username on the e-commerce landing page.

GitHub: <https://github.com/Laz-28/techstore-ecommerce/commit/4bf0f8bdf75f573b4ec1f3bf37b64cf649151260>

<https://github.com/Laz-28/techstore-ecommerce/commit/4bf0f8bdf75f573b4ec1f3bf37b64cf649151260>

Week 4: Server-Side Components and Backend Development Foundations

Required Evidence:

Fig 4.1: Modular Folder Structure.

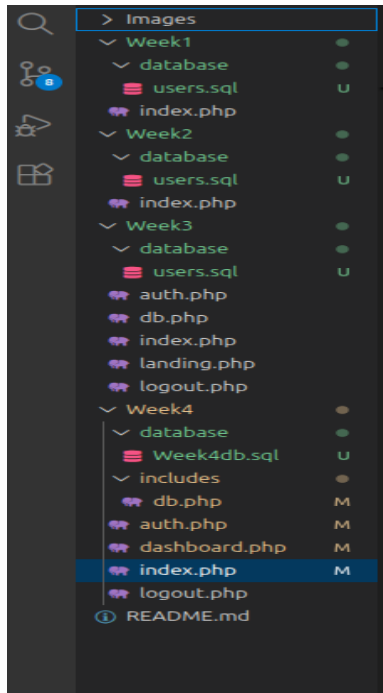


Fig 4.2: PHP Form Processing.

```
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['action']) && $_POST['action'] === 'register')  
    $username = $_POST['username'];  
    $email = $_POST['email'];  
    $password = password_hash($_POST['password'], PASSWORD_DEFAULT);
```

Fig 4.3: Updated Database Connection.

```

<?php
$host = 'localhost';
$dbname = 'Week4db';
$db_user = 'root';
$db_pass = '';

$dsn = "mysql:host=$host;dbname=$dbname;charset=utf8mb4";

try {
    $pdo = new PDO($dsn, $db_user, $db_pass);

    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Database connection failed: " . $e->getMessage());
}
?>

```

Student Reflection: Week 4 transitioned the TechStore architecture from localized scripts to a professionally segregated backend system. By implementing an includes/ directory, the database connection string (db.php) is safely abstracted from the public-facing files. I utilized PHP \$_POST arrays to securely handle form data and mapped the authentication logic to active server sessions (\$_SESSION). Furthermore, I isolated the project state into a specific week4db database to maintain strict version control and rollback capability, adhering to standard DevOps configuration management principles.

Github: <https://github.com/Laz-28/techstore-ecommerce/tree/main/Week4>

Week 5: Database Components and CRUD Operations

Implementing Secure Database Management and Access Control

Required Evidence

Fig 5.1: Database Schema Expansion.

←T→	id	username	email	password	created_at	role
☐ Edit ☐ Copy ☐ Delete	2	Judi	judi@gmail.com	\$2y\$10\$UWOWN1aGQVD1KukXGpkj7.R8C8cwtFqZVynR8p7Qx4...	2026-05-21 07:43:00	customer
☐ Edit ☐ Copy ☐ Delete	3	Mike	mike@gmail.com	\$2y\$10\$OqI3dO5WuPAjDL.1phgZve7N37.0mE9dHdF1fhFtCOM...	2026-05-21 10:37:44	customer
☐ Edit ☐ Copy ☐ Delete	4	Lenny	lenny@gmail.com	\$2y\$10\$4z52QiCcCDX.sL39n4trCujnaD5gMyaaYjyExtlEfg9...	2026-05-27 18:51:10	customer
☐ Edit ☐ Copy ☐ Delete	5	Biden	Biden@gmail.com	\$2y\$10\$82QvuBy9ZiU3X4TGWHKI6O9Kv4gZfXVU2hu5NKbgu.I...	2026-05-28 23:28:34	customer
☐ Edit ☐ Copy ☐ Delete	6	Otis	lazaro@outlook.com	\$2y\$10\$iOSrp58Feh3tsPZ4pwlyx.zP/MLnhjo95C4GBI/Dtyx...	2026-05-29 06:16:08	customer
☐ Edit ☐ Copy ☐ Delete	7	laz	otienolazaro111@gmail.com	\$2y\$10\$rRbDNIWf5dAGR93RJHL2eeCN/GgKYQEX.zN6CI6rtwo...	2026-05-29 06:25:13	admin

←T→	id	name	price	image_url	description	created_at
☐ Edit ☐ Copy ☐ Delete	1	ProBook Workstation	1299.00	wks.webp	High-performance laptop for development and engine...	2026-05-28 23:19:19
☐ Edit ☐ Copy ☐ Delete	2	Lenovo L13 Yoga	459.00	l13.webp	Versatile 2-in-1 laptop with Ubuntu support.	2026-05-28 23:19:19
☐ Edit ☐ Copy ☐ Delete	3	NVIDIA RTX 5090 Ti	2499.00	gpu.webp	Ultimate hardware optimized for high-parameter loc...	2026-05-28 23:19:19
☐ Edit ☐ Copy ☐ Delete	4	Oraimo Freepods Neo	3000.00	oraimo.webp	Noise cancelling ear pods	2026-05-28 23:35:42

Fig 5.2: The CRUD Administrative Panel.

http://localhost/ecommerce_project/Week5/admin_inventory.php

Admin: Inventory Management

[Back to Store](#) [Log Out](#)

Add New Hardware

Hardware Name

Price (Ksh)

Image Filename (e.g. wks.webp)

Description

Save to Database

ID	NAME	PRICE	IMAGE FILE	ACTION
#4	Oraimo Freepods Neo	Ksh 3,000.00	oraimo.webp	Delete
#3	NVIDIA RTX 5090 TI	Ksh 2,499.00	gpu.webp	Delete
#2	Lenovo L13 Yoga	Ksh 459.00	l13.webp	Delete
#1	ProBook Workstation	Ksh 1,299.00	wks.webp	Delete

Fig 5.3: Role-Based UI Rendering.

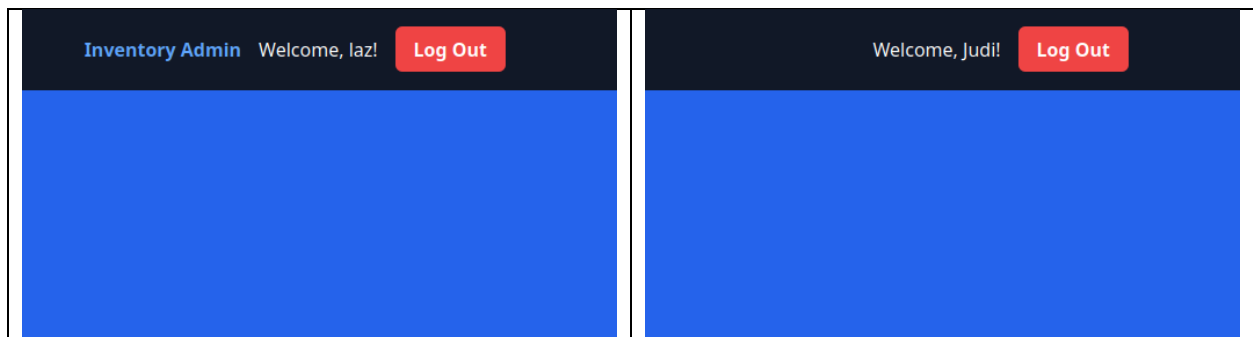


Fig 5.4: Secure Server Logic.

```
1  <?php
2
3  session_start();
4
5  if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
6
7      header("Location: dashboard.php");
8      exit();
9  }
10
11  require 'includes/db.php';
12
13
14  if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['action']) && $_POST['action'] === 'create') {
15      $stmt = $pdo->prepare("INSERT INTO products (name, price, image_url, description) VALUES (?, ?, ?, ?)");
16      $stmt->execute([$_POST['name'], $_POST['price'], $_POST['image_url'], $_POST['description']]);
17      header("Location: admin_inventory.php");
18      exit();
19  }
20
```

Student Reflection:

During Week 5, the TechStore transitioned into a fully database-driven application by implementing robust CRUD protocols and Role-Based Access Control (RBAC). I engineered an administrative interface capable of dynamically managing high-performance hardware inventory, completely replacing hardcoded HTML layouts. To enforce security, I altered the database schema to include user roles and utilized PHP session variables to conditionally render navigation links. Unauthorized access to the admin panel is actively blocked server-side. By executing prepared PDO statements, the system safely processes real-time inventory updates while strictly preventing SQL injection, mirroring industry-grade e-commerce backend standards.

Github: <https://github.com/Laz-28/techstore-ecommerce/tree/main/Week5>

Week 6 : Database Integration and CRUD Operations

Fig 1: Secure Database Connection.

```
1  <?php
2
3  $host = 'localhost';
4  $dbname = 'Week6db';
5  $db_user = 'root';
6  $db_pass = '';
7
8  try {
9      $pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $db_user, $db_pass);
10     |
11     $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
12 } catch (PDOException $e) {
13     die("Database connection failed: " . $e->getMessage());
14 }
15 ?>
```

Fig 2: Create Operation Form.

Admin: Inventory & Reporting

Back to StoreLog Out

Add New Hardware

Hardware Name

Price (Ksh)

Image Filename

Description

Save to Database

Inventory Report Generation

Export to Excel (CSV)

Save as PDF

ID	NAME	PRICE	IMAGE FILE	ACTION
#5	Lenovo L13 yoga	Ksh 40,000.00	l13.webp	EditDelete
#4	Oraimo Freepods Neo	Ksh 3,000.00	oraimo.webp	EditDelete
#1	ProBook Workstation	Ksh 123,399.00	wks.webp	EditDelete

Fig 3: Read Operation Interface.

ID	NAME	PRICE	IMAGE FILE	ACTION
#5	Lenovo L13 yoga	Ksh 40,000.00	l13.webp	EditDelete
#4	Oraimo Freepods Neo	Ksh 3,000.00	oraimo.webp	EditDelete
#1	ProBook Workstation	Ksh 123,399.00	wks.webp	EditDelete

←T→

id

name

price

image_url

description

created_at

☐	EditCopyDelete	1	ProBook Workstation	123399.00	wks.webp	High-performance laptop for development and engine...	2026-05-28 23:19:19
☐	EditCopyDelete	4	Oraimo Freepods Neo	3000.00	oraimo.webp	Noise cancelling ear pods	2026-05-28 23:35:42
☐	EditCopyDelete	5	Lenovo L13 yoga	40000.00	l13.webp	A highly powerful and modern thinkpad laptop built...	2026-06-05 09:44:03

Fig 4: Update and Delete Functionality.

ACTION	
Edit	Delete
Edit	Delete
Edit	Delete

Student Reflection: This week, I successfully integrated our e-commerce web application with a MySQL database, transitioning it from a static interface into a dynamic system. I began by centralizing the database connection into a single configuration file using secure PHP Data Objects. Following this, I developed an administrative dashboard specifically designed to handle foundational CRUD operations. Through this interface, the management team can seamlessly create new hardware listings, read and display current inventory, update existing product details, and delete outdated records. I also implemented prepared statements to ensure all data entry forms are thoroughly protected against SQL injection attacks.